

ZYRA 面向超长程复杂任务的 动态异构多智能体协同系统

参赛技术论文

项目名称 ZYRA 智衍群策

赛题编号 XH-202631

赛题名称 面向超长程复杂任务的动态异构群体智能架构
与深度协同推理技术

2026 年 9 月

摘要

大语言模型智能体已经能够通过工具调用和环境反馈执行多阶段任务，但当任务跨越大量决策、工具调用和进程生命周期时，单纯扩大模型上下文并不能保证目标、约束和中间结果持续一致。与此同时，静态链式或广播式多智能体编排难以适应需求变化和节点故障，还会放大重复通信、错误传播与资源消耗。本文提出 ZYRA，一种面向超长程复杂任务的动态异构多智能体协同系统。系统将大模型的语义提议能力与规范状态的裁决权分离：模型和 Worker 负责理解目标、提出任务分解与执行候选，中心化符号控制平面负责约束检查、版本化提交、资源调度、连续性验证和终止判定。

ZYRA 同时维护逻辑任务图与动态协作图，前者描述必须闭合的任务义务，后者描述当前由哪些异构执行单元承担任务以及信息如何定向流动。协作图通过“语义候选构图、环境条件修正、贡献感知剪枝、硬约束投影”四个环节在线更新。为避免完整对话在智能体之间反复广播，系统以结构化消息传递状态增量、证据和工件引用，并根据下游实际消费回执调整通信关系。系统进一步将不可变事件日志、内容寻址工件、分层记忆和有限模型上下文分离，在压缩、重启、交接和需求修订前后执行连续性验证。面向端、边、云异构环境，ZYRA 先以能力、权限、隐私和资源容量过滤不可行节点，再依据时延、成本、负载和恢复风险进行调度；故障发生后通过租约隔离、代际检查、幂等执行和检查点恢复防止旧结果污染与重复副作用。

在冻结代码与配置下，系统完成了软件工程交付和跨源技术研究两个不同领域的封存长程任务。独立验证器从原始规范事件重算得到 2,295 次和 7,168 次有效状态迁移，两次运行均经历 5 类故障注入，人工干预数、无效迁移数和阻塞项均为 0，最终工件通过独立验证。上述结果证明了系统在指定跨领域实例中实现无人干预闭环的可行性，但不被用于推导尚未完成的统计显著性或跨任务普遍优势。

关键词：大语言模型智能体；多智能体系统；动态协作拓扑；长程记忆；神经符号协同；故障恢复；端边云协同

目录

1 绪论	6
1.1 从工具调用走向超长程自治.....	6
1.2 三类相互耦合的困难	6
1.3 核心洞察	7
1.4 主要贡献	7
2 相关研究与技术缺口	7
2.1 长程语言智能体	7
2.2 多智能体编排与通信拓扑.....	8
2.3 外部记忆与上下文压缩	8
2.4 神经符号与端边云协同	9
2.5 本文位置	9
3 问题定义与总体架构	9
3.1 超长程复杂任务	9
3.2 任务图与协作图	9
3.3 可信系统状态	10
3.4 硬约束与软目标	10
3.5 总体架构	11
3.6 端到端控制闭环	11
4 动态异构协作方法	12
4.1 方法概览	12
4.2 语义候选构图	13
4.3 环境条件修正	13
4.4 贡献感知剪枝	13
4.5 约束投影与版本化提交	14
4.6 算法流程与复杂度	14

5 长程通信与记忆连续性.....	15
5.1 义务定向的结构化通信	15
5.2 工件外置与消费证明	16
5.3 分层记忆与事实边界	16
5.4 受保护的上下文压缩	17
5.5 连续性验证门	17
6 异构调度与故障恢复	17
6.1 从逻辑义务到物理执行	17
6.2 硬约束过滤与软排序	18
6.3 租约 代际与幂等	18
6.4 故障分类与恢复	18
6.5 恢复成功条件	19
7 系统实现与用户交互	19
7.1 可部署系统结构	19
7.2 异构 Worker 与能力扩展	20
7.3 CLI 与 Web 的协同入口.....	20
7.4 证据驱动的完成判定	21
8 实验设计与验证结果	21
8.1 评价原则	21
8.2 研究问题	21
8.3 场景与冻结协议	21
8.4 封存长程运行结果	22
8.5 软件工程案例分析	22
8.6 跨源研究案例分析	22
8.7 指标与对照计划	23
8.8 结果边界	23
9 应用价值与可扩展性	23
9.1 面向复杂知识工作的数字劳动力.....	23

9.2 用户体验与治理	24
9.3 社会与经济价值	24
10 局限与结论	24
10.1 当前局限	24
10.2 结论	25
参考文献	26
附录 A 核心算法与复杂度补充.....	27
A.1 任务完成门.....	27
A.2 通信指标.....	27
A.3 记忆检索.....	27
A.4 恢复不变量.....	27
附录 B 工程实现与部署边界.....	28
B.1 产品入口	28
B.2 运行时职责	28
B.3 部署要求.....	28
B.4 技术细节披露边界.....	28
附录 C 证据与复现说明.....	28
C.1 权威封存证据.....	28
C.2 实验复现原则.....	29
C.3 后续对照实验.....	29

1 绪论

1.1 从工具调用走向超长程自治

大语言模型正在从一次性文本生成器转变为能够观察环境、调用工具并根据反馈调整行为的任务智能体。**ReAct** 将语言推理与环境行动交错组织，**Reflexion** 通过语言化反馈改进后续尝试，**Voyager** 则展示了自动课程、技能库与执行反馈在持续探索中的作用。这些工作奠定了“推理、行动、观察”闭环的基本范式。然而，真实的软件工程、跨源研究和基础设施运维并不是若干轮对话的简单延长：任务会持续数小时甚至更久，需求和资源可能变化，部分工具调用具有不可逆副作用，最终结果还必须由文件、测试、来源或业务规则验证。

这类任务的关键问题不是模型能否生成一个看似合理的下一步，而是系统能否在很长的运行过程中持续回答三个问题：当前可信状态是什么，下一步允许执行什么，以及何时能够证明任务已经完成。如果答案只存在于某个模型的私有对话中，上下文压缩、进程重启、角色交接和需求修订都可能使任务状态发生分叉。一个语义流畅的总结也可能保留旧需求、遗漏未完成义务，或忘记某项外部操作是否已经生效。

因此，超长程自治首先是一个运行时状态管理与协作控制问题，其次才是上下文长度问题。系统既要利用大模型理解开放语义，又要以可重放、可验证的机制约束状态变更。

1.2 三类相互耦合的困难

第一类困难是状态和记忆连续性。完整历史无法无限注入每一轮模型上下文，只保留最近消息又会遗失长期目标、需求版本、失败原因和关键证据。如果摘要成为唯一事实来源，一次错误压缩就可能永久改变后续执行方向。

第二类困难是组织和通信。固定顺序链在节点故障时缺少替代路径，固定星型容易形成中心瓶颈，全连接或广播通信的潜在消息边数量随智能体规模近似平方增长。更严重的是，未经筛选的错误或过期信息会传播给与当前义务无关的节点。

第三类困难是异构资源与恢复。端侧、边缘和云端节点在能力、隐私边界、时延、成本和故障域上不同。预先生成的计划会因网络分区、工具失败或 **Worker** 重启而失效；简单重试还可能重复已经发生的写入、发布或资源消耗。

这三类困难相互影响。拓扑变化会改变信息传播路径，记忆缺失会导致错误调度，故障恢复又要求准确知道已完成义务和未确认副作用。因此，不能依靠彼此独立的脚本分别处理，而需要统一的状态与控制平面。

1.3 核心洞察

ZYRA 的核心洞察是将“语义提议”与“状态裁决”分离。大模型擅长理解高维模糊意图、提出角色和行动候选、解释异常并生成业务交付物；确定性运行时更适合维护版本、权限、预算、租约、图不变量、幂等和终止条件。二者的关系不是用规则替代模型，而是让模型的开放语义能力在明确可行域内发挥作用。

$$p(t) \sim P\theta(\cdot | X_{obs}(t), K(t)), \quad u(t) = \Pi[F(X_{obs}(t))](p(t)) \quad (1)$$

模型或 Worker 提出的候选不能直接覆盖主状态。控制平面读取版本绑定的可信快照，检查能力、权限、隐私、资源、预算、图结构、记忆与证据约束；合法候选被转换为可提交动作，非法候选得到带原因的拒绝、降级或重新规划事件。这里的“中心化”指状态权威和控制平面中心化，不表示由一个中心大模型完成所有推理，也不要求收集 Worker 的完整思维过程。

1.4 主要贡献

本文的主要贡献包括三个方面。

第一，提出受约束的动态异构协作图控制方法。系统区分相对稳定的任务义务图和随环境变化的执行协作图，并通过语义构图、环境修正、贡献剪枝和符号投影在线更新执行关系。

第二，提出可验证的长程状态与记忆连续性机制。系统以不可变事件和工件作为事实边界，以工作、情景、语义和技能记忆提供可消费视图，并通过受保护压缩、按需检索和连续性门控制关键事实进入下一轮推理。

第三，实现支持异构调度、故障恢复和独立验证的可部署系统。CLI 与 Web 共用同一规范状态，代码、浏览器、模型和领域 Worker 通过统一任务与证据协议协作；系统能够接受需求变化和节点失效，并在两个跨领域封存任务中完成无人干预闭环。

2 相关研究与技术缺口

2.1 长程语言智能体

ReAct 将推理和行动交错生成，使模型能够根据工具观察调整后续步骤[1]；Reflexion 将反馈保存为语言化经验，用于改进后续尝试[2]；Voyager 通过自动课程、技能库和环境反馈展示了持续探索能力[3]。这些研究主要回答局部智能体如何选择下一步、如何从反馈中改进。ZYRA 关注的是多个局部循环如何共享唯一、持久且可恢复的任务状态，二者是互补关系。

只扩大模型上下文也不能等价地扩大可靠任务时长。历史越长，完整注入成本越高；摘要越激进，目标与约束越容易丢失。长程系统还必须在进程重启、Agent 交接和需求修订后识别事实版本与未完成义务，这超出了单轮提示组织的范围。

2.2 多智能体编排与通信拓扑

AutoGen 等框架提供了组合对话智能体的通用抽象[4]，GPTSwarm 将语言智能体表示为可优化计算图[5]。ARG-Designer 联合生成角色和通信结构[6]，CARD 根据环境条件设计拓扑[7]，AgentPrune 则从时空通信图中剪除冗余边[8]。这些工作证明了任务条件构图、环境适配和边级优化的价值。

现有方法通常以一次任务的最终正确率为主要目标，并假设智能体、工具和资源在评估期间相对稳定。超长程运行还需要解决候选拓扑是否满足权限和隐私、多个分支如何基于同一版本提交、节点故障后如何恢复、通信是否实际支持下游验证等问题。ZYRA 将相关思想工程化为确定性、有限且可重放的在线策略；本文不声称复现原方法的训练过程，也不继承其未经本系统实验验证的性能结论。

表 1 典型协作方式与 ZYRA 的研究位置

方法类型	主要优势	超长程环境中的不足
单智能体	状态集中、实现简单	能力边界单一，长上下文易产生遗忘与漂移
静态链式协作	流程清晰、通信成本可控	节点失效或需求变化后缺乏替代路径
静态星型协作	便于集中协调	中心负载高，结构不能随任务阶段改变
全连接或广播	信息覆盖广	通信近似平方增长，错误与冗余容易扩散
学习式拓扑	能够按任务分布优化结构	通常缺少运行时硬约束、状态版本和恢复语义
ZYRA	动态、受约束、可恢复且可审计	当前软评分仍依赖启发式配置，需要更大规模在线评估

2.3 外部记忆与上下文压缩

RAG 将参数化生成与可检索外部知识结合[9]，Generative Agents 以相关性、时近性和重要性组织经历[10]，MemGPT 借鉴分层存储管理有限上下文[11]，LLMLingua 通过预算控制压缩提示[12]。这些方法主要优化“给模型看什么”，而长程运行时还需要判断“该事实当前是否有效”。

ZYRA 因此把不可变事件、工件、记忆视图、检索结果和模型上下文视为不同对象。压缩只改变有限上下文，不删除事实账本；关键事实还要经过来源、版本、冲突和取代关系检查。

2.4 神经符号与端边云协同

LLM+P 和 LINC 展示了语言模型负责语义解析、符号系统负责可判定规划或推理的互补路线[13,14]。分布式快照研究则说明，在异步系统中记录一致全局状态本身就是独立问题[15]。在异构推理方面，Neurosurgeon 和 Edgent 研究了终端、边缘与云之间的模型切分和协同推理[16,17]。

ZYRA 将这些原则扩展到多智能体运行时：调度对象不仅是模型层或算子层，还包括角色、Worker、工具和物理位置；符号层不仅检查一次计划，还持续裁决版本、权限、资源、恢复和终止。

2.5 本文位置

本文的创新重点不是把若干现有名称简单组合，而是为它们建立共同的可信状态边界。动态拓扑在同一版本快照上生成，通信受任务义务和预算约束，记忆必须在压缩和恢复后继续可验证，任何候选变更均由唯一控制平面提交。这使系统能够在开放环境中持续调整执行结构，同时避免多份主状态、非法迁移和重复副作用。

3 问题定义与总体架构

3.1 超长程复杂任务

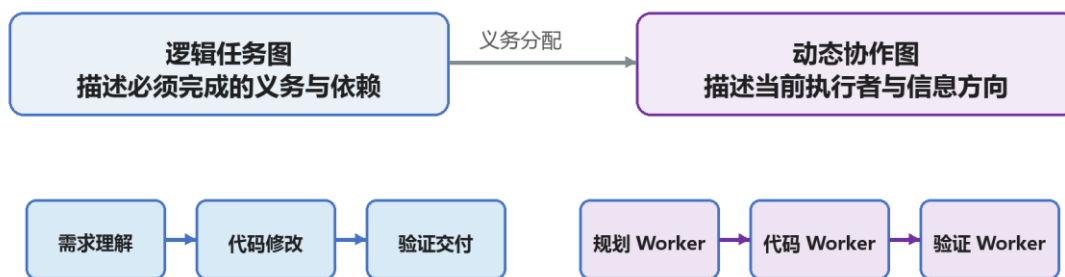
设系统接收自然语言目标，并在异构模型、Agent、工具和端边云节点之间持续进行分解、调度、执行、验证与恢复。任务可能跨越多个模型轮次和进程生命周期，需求、资源与外部事实也会变化。系统的目标不是生成一次回答，而是产生通过验证的最终交付物及其证据链。

为便于理解，可以考虑一个软件交付任务：系统需要在隔离工作区修改 SDK，执行期间用户更新需求，代码 Worker 失联，边缘节点网络中断。可靠系统必须保留原始目标、当前需求版本、已完成修改、测试结果和未确认副作用，在合法的新节点上继续工作，并证明最终补丁满足修订后的要求。

3.2 任务图与协作图

ZYRA 同时维护两类图。逻辑任务图描述任务义务及其依赖，回答“哪些工作必须完成，以及先后关系是什么”；动态协作图描述当前激活的角色、Worker 和有向通信关系，回答“现在由谁执行，结果交给谁”。

同一目标 两类图承担不同职责



执行节点可以替换 业务义务与完成标准保持

需求变化显式更新任务图 环境变化只调整协作图

图 1 逻辑任务图与动态协作图的分离

两张图通过义务到 Worker 的分配关系连接，但不能混用。执行者发生变化时，业务义务仍然保持；需求修订时，旧义务被显式标记为取代，而不是从对话中静默消失；协作图则可以根据节点健康和资源变化重新构造。

3.3 可信系统状态

$$X(t) = (D(t), G(t), O(t), M(t), H(t), B(t), Q(t), A(t))$$

(2)

其中，D 表示任务图，G 表示协作图，O 表示未闭合义务，M 表示经验证记忆，H 表示节点健康与容量，B 表示预算，Q 表示权限、隐私和业务约束，A 表示工件与证据。控制器实际读取带时间戳、来源和版本的可观测快照，而不是假定环境状态完全可知。

系统状态由规范事件折叠得到。模型对话、临时分支和 Worker 私有缓存可以辅助计算，但不构成唯一事实来源。任何状态变更都需要绑定基础版本、因果关系和幂等标识，因此系统能够在重启后从同一事件前缀恢复一致状态。

3.4 硬约束与软目标

合法动作必须同时满足任务依赖、能力、权限、隐私、资源、预算、图结构、版本、记忆和证据约束。硬约束之间不能相互补偿：隐私不允许的数据不能因为云节点更快而上传，基于旧版本的拓扑也不能因为预计收益更高而覆盖当前状态。

在可行候选内部，系统再权衡任务推进、验证价值、通信、时延、切换成本和失败风险。

$$\max_{u \in F(X_{\text{obs}}(t))} \Delta U_{\text{obligation}} + \lambda_v \cdot U_{\text{verify}} - \lambda_c \cdot C_{\text{comm}} - \lambda_l \cdot C_{\text{latency}} - \lambda_s \cdot C_{\text{switch}} - \lambda_f \cdot R_{\text{failure}}$$

(3)

该目标描述滚动控制的方向，不代表系统求得全局最优解。真实环境的未来变化不可完全预知，ZYRA 在每个可信快照上执行有界决策，观察真实结果后再进入下一轮。

3.5 总体架构

中心化状态控制 去中心化异构执行

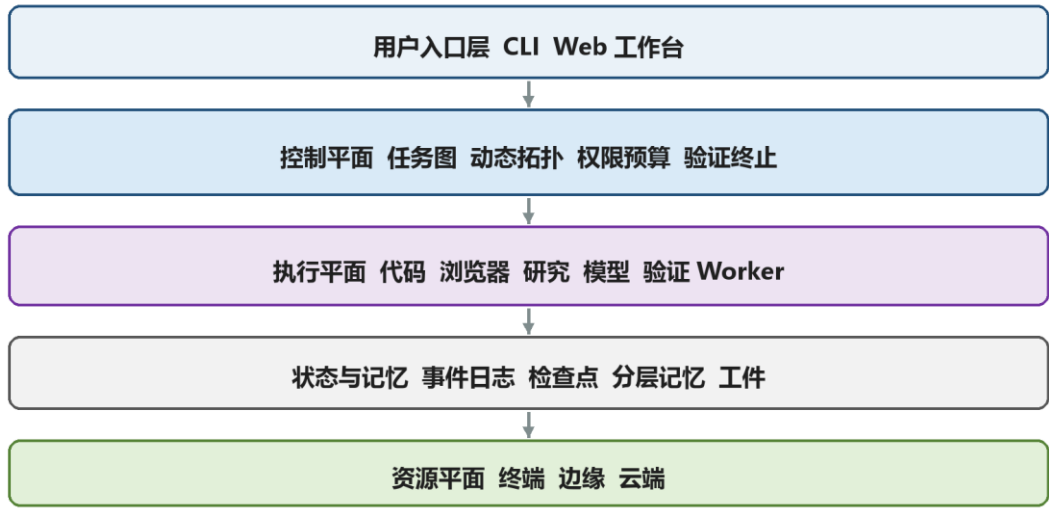


图 2 ZYRA 中心化控制与去中心化执行架构

系统由五个相互连接的层次组成。用户通过 CLI 或 Web 提交目标、查看进度和处理必要权限；控制平面维护任务图、拓扑、约束、预算与验证；执行平面包含代码、浏览器、研究、模型等异构 Worker；状态与记忆平面保存事件、检查点、工件和记忆视图；资源平面提供本地、边缘与云端执行位置。

控制平面拥有规范状态的写权限，Worker 只提交候选动作、局部事件和工件。Web 与 CLI 是同一后端状态的不同投影视图，不形成第二套 Agent 运行时。

3.6 端到端控制闭环

算法 1 ZYRA 超长程控制循环

- 1 接收目标、预算、权限和部署约束，创建规范任务状态
- 2 将目标规范化为可验证义务及其依赖
- 3 读取版本绑定的可信状态快照

- 4 必要时由模型提出任务、角色、工具或恢复候选
- 5 构造候选协作图，并根据当前环境修正和剪枝
- 6 执行能力、权限、隐私、预算、租约和图结构检查
- 7 原子提交合法的拓扑与状态增量
- 8 将就绪义务调度给选定 Worker
- 9 追加执行事件、工件、证据和资源回执
- 10 验证义务完成、关键记忆和未确认副作用
- 11 可恢复故障从最近可信快照重新路由或规划
- 12 全部必要义务通过验证后生成最终交付，否则保留可恢复终态

4 动态异构协作方法

4.1 方法概览

静态拓扑在环境稳定时容易实现，但不能同时处理任务变化、节点失效和通信预算。让大模型直接输出任意通信图虽然灵活，却难以稳定满足权限、隐私、无环性和版本一致性。ZYRA 将图更新分解为四个职责明确的环节。

动态协作图只提交通过约束的状态增量



软评分不能覆盖权限 隐私 容量 预算 版本与图结构等硬条件

同一快照产生候选 同一提交序列形成可重放的规范拓扑

图 3 动态异构协作图的在线更新流程

$G(t,0) \rightarrow \text{semantic } G(t,1) \rightarrow \text{condition } G(t,2) \rightarrow \text{prune } G(t,3) \rightarrow \text{projection } G(t+1)$

(4)

语义候选构图回答“任务需要谁”；环境条件修正回答“当前谁能够可靠执行”；贡献感知剪枝回答“哪些通信关系值得继续保留”；约束投影和图状态托管决定候选是否能够成为规范状态。

4.2 语义候选构图

候选构图从未完成义务和能力目录出发，同时选择角色、执行节点及必要入边。系统先排除能力或基础权限不匹配的候选，再通过有界搜索构造能够覆盖关键义务的较小团队。候选得分抽象为义务覆盖、能力匹配、依赖可达性、关键路径和恢复价值的正向收益，减去通信和切换成本。

$$sARG = w_o \cdot c_o + w_a \cdot c_a + w_d \cdot c_d + w_k \cdot c_k + w_r \cdot c_r - w_m \cdot c_m - w_s \cdot c_s \quad (5)$$

当前实现使用确定性排序和有界束搜索，使相同快照与配置产生可重放结果。具体权重、候选上限和束宽属于工程参数，放入内部配置而不在正文展开。该方法不被描述为训练得到的最优策略。

4.3 环境条件修正

语义上合适的协作边并不一定在当前环境中可执行。例如，目标 Worker 可能失联，边缘节点可能负载过高，某类数据也可能禁止离开本地。环境修正模块读取节点健康、容量、负载、时延、成本、部署位置、隐私适配和遥测新鲜度，对每条有向候选边进行残差修正。

权限不足、位置非法、节点不可用和容量不足属于硬拒绝，不参加软评分。对合法边，系统比较健康、时延、成本和负载等因素；新增边和已有边使用不同阈值，并要求非故障变更满足最短驻留和连续确认，从而避免因单次负载尖峰频繁增删同一条边。

$$sCARD(e) = w^T \varphi(t, e) / \sum_j w_j - \kappa(e), \quad H(t, e) = 0 \Rightarrow \text{reject / drop} \quad (6)$$

该模块只修正上游候选图或受约束替代边，不能绕过任务义务自行重建任意拓扑。

4.4 贡献感知剪枝

环境可用不等于通信有效。ZYRA 根据滑动窗口中的真实交付回执评估边贡献，包括消息送达、证据消费、工件贡献、验证通过、重复、失败、重试和资源占用。高贡献边优先保留，长期低价值或高风险边进入删除候选。

剪枝必须保护关键路径、唯一证据、未闭合义务、恢复、连续性和验证相关边。若只保留这些受保护关系仍超过通信预算，系统报告预算不可行，而不是静默删除完成任务所需信息。

$$\min \sum_e k_e \cdot C_e - \lambda \sum_e k_e \cdot s_e, \quad \text{s.t. } k_e = 1 \text{ for every protected edge } e$$

(7)

这种设计避免通过“少发必要信息”获得表面的低通信量。通信节省必须与义务覆盖、证据利用和最终验证同时报告。

4.5 约束投影与版本化提交

三个阶段只产生候选增量，不直接修改规范图。投影器检查基础快照、机制就绪状态、角色和能力注册、权限、隐私、容量、租约、预算、扇出、驻留、边端点和有向环。候选通过检查后，由图状态托管模块基于当前修订执行原子提交。

并行 Worker 可以从同一快照独立计算候选，但规范图保持单写者语义。相同幂等标识和输入摘要的重复提交返回原回执；可安全重基的陈旧候选重新计算；读写、写写或结构冲突触发拒绝和重新规划。

4.6 算法流程与复杂度

算法 2 动态协作图更新

- 1 从未完成义务、角色目录和当前图构造策略快照
- 2 过滤能力、注册和基础权限不满足的角色
- 3 通过有界搜索生成覆盖关键义务的基础候选图
- 4 根据健康、负载、时延、成本、隐私和新鲜度修正有向边
- 5 根据交付、证据消费、验证和失败历史计算边贡献
- 6 标记关键路径、唯一证据、恢复和验证边为受保护关系
- 7 在通信预算内删除低贡献非保护边
- 8 预览候选增量并执行全部硬约束检查
- 9 通过版本检查后原子提交，否则拒绝或重新规划

表 2 核心运行时阶段的复杂度

阶段	主要计算	时间复杂度	受控规模
语义候选构图	角色、节点和边的有界联合搜索	$O(BKN + BKN \log(BN))$	候选数、束宽和最大节点数
环境条件修正	边级遥测编码与残差计算	$O(E)$	候选边数
贡献感知剪枝	回执聚合与边效用排序	$O(R + E \log E)$	回执数与候选边数
约束投影	权限、资源、端点与图不变量	$O(V + E + C)$	节点、边和约束数
调度与接管	Worker 过滤与稳定排序	$O(W \log W)$	注册 Worker 数

5 长程通信与记忆连续性

5.1 义务定向的结构化通信

多智能体通信成本不仅来自消息数量，也来自重复上下文、不可验证结论以及下游不会消费的证据。若 n 个 Agent 采用全连接广播，单轮潜在有向边数量为 $n(n-1)$ 。ZYRA 不允许 Worker 默认广播完整对话，而是在已经提交的协作图上，向能够推进或验证当前义务的下游发送结构化交付包。

消息包包含有界摘要、局部约束、状态差量、证据引用、工件引用、来源和有效期。消息本身不等于状态提交；其中的状态差量仍要经过对应投影器。中心控制平面负责路由与状态裁决，但不需要用另一个大模型重新理解每条消息，也不收集完整思维链。

传递结构化交付 不广播完整对话

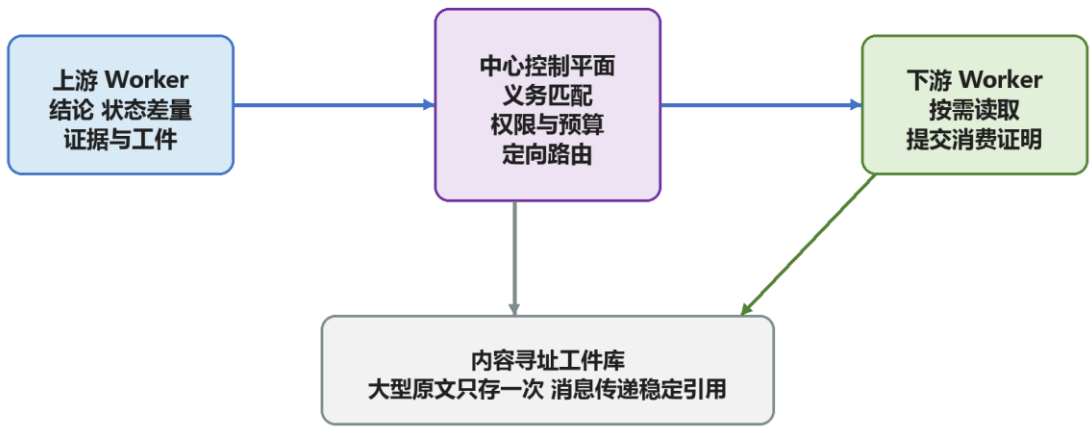


图 4 义务定向的结构化通信与工件外置

5.2 工件外置与消费证明

代码补丁、网页原文、测试日志和大型研究材料只保存一次。消息传递摘要、内容摘要值和稳定引用，下游在具备权限时按需读取。这样既减少重复传输，又保留独立验证所需原文。

系统不仅记录“消息已经发送”，还记录下游实际消费了哪些证据、生成了哪个决策或工具调用、产生了哪些工件。真实消费结果会反向更新协作边贡献，为下一轮剪枝提供依据。

通信效果使用可观测指标评价，包括传输字节、Token、重复率、证据利用率和单位验证义务成本。除非明确计算消息接收者分布上的 Shannon entropy，正文不使用未经测量的“熵下降百分比”。

5.3 分层记忆与事实边界

ZYRA 不把模型当前看到的文本视为完整任务记忆。不可变事件日志和内容寻址工件构成持久事实边界，系统从中派生四类记忆视图。

事实账本与模型上下文分离

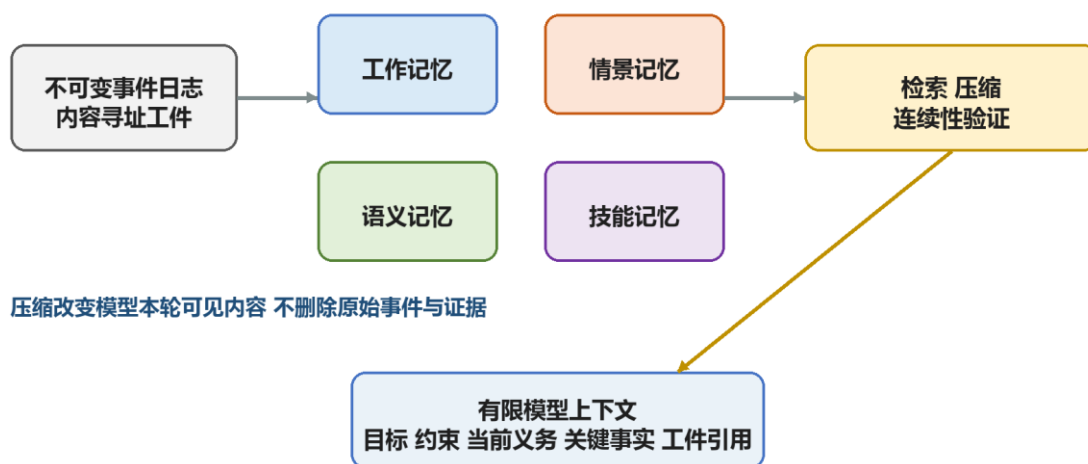


图 5 从事实账本到有限模型上下文的记忆连续性

工作记忆保存当前目标、预算、节点状态和近期决策；情景记忆保存事件、失败和恢复轨迹；语义记忆保存稳定事实、约束和工件知识；技能记忆保存可复用流程、运行条件与历史结果。记忆记录携带来源、版本、时间、证据与工件引用，而不是无来源的自由文本摘要。

对于当前义务，检索器可以组词法检索、可选向量检索、重要度与新鲜度排序，并通过多样性重排降低近重复结果。索引不可用或摘要不一致时，系统显式退化或触发修复，检索索引不能取得事实所有权。

5.4 受保护的上下文压缩

当模型上下文接近预算时，系统压缩推理视图而不是删除事件历史。初始目标、当前需求版本、未完成义务、关键决策、失败与恢复、最近状态以及完整的工具调用与结果组属于受保护内容。过长原文被外置为工件，压缩摘要只保留稳定引用。

$$(K\text{tilde}(t), A\text{plus}(t)) = C(K(t), \Gamma(t); P(t), B\text{ctx}(t)), \quad L'(t) = L(t) \quad (8)$$

如果存在尚未闭合的工具调用，系统推迟压缩，避免调用和结果被拆开。连续压缩失败会触发熔断并保留可诊断状态，而不是不断重写上下文。

5.5 连续性验证门

压缩成功只说明文本变短，并不证明任务语义保持。ZYRA 在压缩恢复、Worker 交接、节点替换、进程重启和需求修订等边界前后比较连续性快照。未完成义务必须保留或被新需求明确取代；关键事实必须具有有效来源、内容摘要和版本，且不能处于陈旧、冲突或已取代状态。

$$\begin{aligned} \text{Gate}(C\text{minus}, C\text{plus}) = 1 \Leftrightarrow & \text{Oopen}(\text{minus}) \subseteq \text{Oplus} \cup \text{Osuperseded} \\ & \wedge \text{ every } f \in F\text{critical}(\text{plus}) \text{ satisfies Safe}(f) \end{aligned} \quad (9)$$

连续性验证不仅发生在执行前。下游完成工作后还要提交消费证明，说明实际读取了哪些关键事实，并将其绑定到决策、工具调用和工件。如果执行使用旧需求、关键事实未被消费或已经完成的工件被重复生成，最终连续性回执失败并触发恢复或重新规划。

6 异构调度与故障恢复

6.1 从逻辑义务到物理执行

动态协作图决定哪个角色应承担义务，但逻辑分配不等于可靠的物理执行。端侧、边缘和云端节点在能力、隐私边界、容量、时延、费用和故障域上不同，同一 Worker 标识在进程重启后也可能代表新的物理实例。

ZYRA 将逻辑义务、物理尝试和资源租约分离。一个义务可以在故障后产生新的物理尝试，但业务任务只保留一份。后继尝试记录父尝试、恢复原因和新的执行身份，避免恢复过程中复制任务或错误复用旧失败结果。

6.2 硬约束过滤与软排序

调度首先检查能力、工具、部署位置、权限、隐私、容量和健康状态。只有合法候选进入软排序，再依据能力匹配、时延、负载、成本与故障历史选择执行位置。

$$\begin{aligned} w^*(i) &= \arg \max_j s(i,j), \quad \text{where } \chi(i,j) = 1 \\ w(j): \chi(i,j) &= 1 \Rightarrow \text{capability, permission, privacy, capacity, health valid} \end{aligned} \quad (10)$$

该顺序确保敏感数据不会因云端速度更快而越过位置约束，也确保 edge-only 或 local-only 任务不会在故障后静默迁移到非法位置。多 Agent 并行度同时受任务依赖、Worker 并发槽和资源容量限制，不进行无界扇出。

6.3 租约 代际与幂等

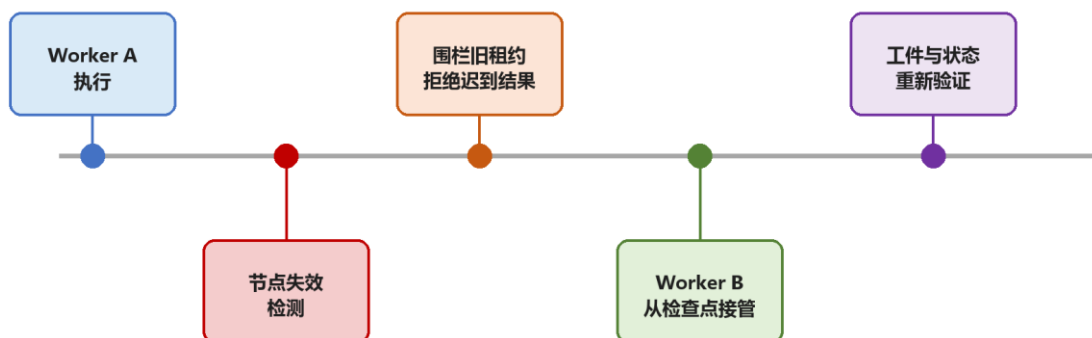
调度决定回答“选谁”，租约才授予一次物理尝试在限定时间内执行和提交的权利。租约取消、过期或被接管后，旧执行者即使恢复连接，其迟到结果也不能进入规范状态。Worker 重启会产生新的进程代际，只有重新登记能力并获得新租约的实例才能接管任务。

幂等标识用于判断同一逻辑操作是否已经执行。相同输入的重复请求返回原回执；标识相同但输入不同则报告冲突；恢复后的新物理尝试获得新的绑定标识。对于“外部操作可能已发生但响应丢失”的情况，系统先查询回执或执行专门验证，不能直接重放。

6.4 故障分类与恢复

系统将工具超时、Worker 不可用、浏览器崩溃、权限拒绝、模型或结构错误、验证失败、节点失效和需求漂移规范化为故障信号。根据故障可恢复性和当前约束，系统选择重试、重新路由、后端回退、检查点恢复、重新规划、安全降级或带原因终止。

恢复不是简单重试 而是重新建立合法提交权



能力 位置 隐私和未确认副作用约束在接管前后保持

图 6 Worker 失效后的围栏 接管与重新验证

Worker 丢失时，控制平面先围栏化旧租约，再从规范存储读取原能力、位置和未确认副作用要求。系统优先寻找满足原约束的替代 Worker，创建新的物理尝试，从可信检查点恢复上下文，执行后重新验证工件和回执。如果不存在合法替代者，系统进入可解释的重新规划或阻塞状态，而不是放宽隐私与权限条件。

6.5 恢复成功条件

$$\text{Recovered}(\text{anew}) \Leftrightarrow \text{LeaseValid}(\text{anew}) \wedge \text{ContinuityGate}(\text{anew}) \wedge \text{ArtifactVerified}(\text{anew}) \wedge \text{PendingEffects}(\text{anew}) = \emptyset$$

(11)

恢复成功要求新尝试拥有有效租约，通过记忆连续性检查，工件经过验证，且不存在未确认副作用。该定义把“进程重新启动”和“任务正确恢复”区分开来。

7 系统实现与用户交互

7.1 可部署系统结构

ZYRA 是由 Python 与 TypeScript 共同实现的本地优先系统。Python 侧维护任务、事件、拓扑、记忆、调度、恢复和 API 状态；TypeScript 侧承载 CLI、Web、模型 Provider 控制及代码 Worker 的模型工具循环。两种语言通过类型化 API、事件和 Worker 协议连接，不形成两套对等编排系统。

表 3 可部署系统的主要组成

组成	主要职责	面向用户或系统的结果
CLI	创建、控制、恢复和非交互执行任务	长程任务默认入口
Web 工作台	观察拓扑、事件、权限、记忆、工件和验证	可视化治理与证据查看
API 服务	托管任务、会话、事件和规范状态接口	CLI 与 Web 的共同事实来源
异构 Worker	执行模型推理、工具调用和领域任务	结构化交付、工件与局部回执
状态与记忆	事件、检查点、记忆视图和工件存储	可重放历史与有限上下文
验证与恢复	完成谓词、连续性、租约和故障闭合	可信终止或可恢复失败记录

系统构建提供依赖锁定、健康检查、发布审计、干净安装验证和回滚机制。部署者配置模型 Provider 凭据后，可通过 CLI 直接执行任务，也可启动 Web 工作台观察同一任务的拓扑、事件、权限、记忆、工件和验证状态。

7.2 异构 Worker 与能力扩展

Worker 层按照能力而不是固定模型名称注册。当前系统具备代码执行、浏览器操作、记忆整理、研究、验证和模型推理等边界，并允许通过技能和 MCP 扩展工具能力。调度器以能力、位置、权限和资源清单匹配义务，因此新增或替换 Worker 不需要重写整个任务流程。

系统支持多个模型 Provider。模型负责开放语义任务，工具和符号组件负责可确定验证的状态变化。模型调用失败时，后端回退和重新路由仍要遵守原预算与权限，不会产生隐藏的第二套主状态。

7.3 CLI 与 Web 的协同入口

CLI 是长程任务的默认执行入口，支持创建、继续、恢复、取消、重定向和非交互运行。Web 工作台用于展示完整拓扑、因果事件、工件、验证、权限和故障恢复信息。两者连接同一 API 服务和规范任务状态：退出界面不会自动终止后台任务，再次连接可以从事件游标和状态快照恢复。

普通用户界面优先展示目标、回答、少量执行活动、文件变更、验证和最终结果；原始事件、内部标识和大型诊断信息放在可展开的证据视图中。这种分层既降低使用门槛，又保留比赛评审和系统审计所需细节。

7.4 证据驱动地完成判定

任何 Worker 的“任务已完成”文本都不能直接结束任务。完成门检查当前需求版本下的必要义务、预期工件、测试或领域验证、关键记忆连续性和开放租约。最终结果绑定工件摘要、验证器回执和因果事件索引；即使任务失败，系统也保留最后可信状态、已验证工件、未完成义务和恢复条件。

8 实验设计与验证结果

8.1 评价原则

实验区分三类证据。单元和集成测试用于证明实现不变量受到覆盖；封存运行用于证明真实任务路径存在；只有在相同任务、模型、预算和故障计划下重新执行的基线与消融，才能支持方法间的统计比较。测试通过数量不能直接当作任务成功率，单次运行也不能推导跨任务普遍优势。

本版论文采用已经封存并通过独立验证的真实任务结果，拒绝把历史演示数字、预录轨迹或模拟回执写成在线实验结论。由于现有完整在线基线样本尚不足，正文报告存在性与机制验证，不使用“统计显著提升”等表述。

8.2 研究问题

实验围绕四个问题组织：第一，系统能否完成跨领域长程任务并产生可验证交付；第二，动态拓扑和义务定向通信能否在保持关键路径的同时控制冗余；第三，需求变化、压缩、重启和节点故障后能否保持状态连续并避免重复副作用；第四，端边云调度能否在能力、隐私、容量和成本约束下选择合法位置。

8.3 场景与冻结协议

软件工程场景要求系统在隔离工作区完成 SDK 修改，处理中经历需求变化、测试或执行故障、Worker 丢失和边缘网络变化，最终交付补丁、测试证据和摘要绑定报告。跨源技术研究场景要求系统从多个实时公开来源取得原始内容，区分事实与建议，处理来源超时、研究节点失效、限流和网络变化，最终交付逐项可追溯的结构化报告。

每次封存运行绑定任务输入、代码版本、模型、预算、故障计划、拓扑策略、硬件网络环境和独立验证器。原始事件、状态前后摘要链、工件、租约、资源决策和验证结果保留在证据包中。独立验证器不信任运行器自报步数，而是排除心跳、日志、界面刷新、重放和无状态变化事件后重新计算有效迁移。

8.4 封存长程运行结果

表 4 P2-S06-02 attempt 08 封存运行结果

场景	领域	有效迁移	故障注入	人工干预	无效迁移	最终验证
software-sdk	软件工程交付	2,295	5	0	0	通过
technical-intelligence	跨源技术研究	7,168	5	0	0	通过

两个任务均在冻结条件下完成，失败运行数为 0，人工干预数为 0。每个任务经历 5 个注入故障，并记录压缩恢复、进程重启、Worker 交接和需求修订的连续性回执。关键事实召回率和未决义务保持率为 1.0；过期、冲突和污染记忆被拒绝；权限、隐私和未确认副作用的对抗候选未产生不安全提交。

两次运行均形成真实的本地、边缘和云端派发回执。边缘执行进程在回执提交后被停止，系统将后续工作安全回退到满足约束的本地执行位置，工件连续性保持。动态图中角色和执行单元发生实际增删，最终变更由图状态托管模块提交。

有效迁移数量表示独立验证器确认的规范状态变化，包括工具、工件、恢复、路由、验证和其他具有语义效果的事件，不等同于大模型推理次数。该口径避免使用日志数量或界面刷新制造“长程步数”。

8.5 软件工程案例分析

软件工程任务共产生 2,295 次有效迁移，其中包括 1,118 次工具效果、1,123 次工件效果、12 次压缩恢复效果、7 次位置选择、7 次恢复、6 次路由和 8 次验证效果。任务执行期间发生需求变化和执行节点故障，旧执行身份被隔离，替代路径从可信状态继续工作，最终补丁及其验证摘要被绑定到规范任务。

该案例说明，长程任务的关键并不是保持一段不断增长的聊天记录，而是让每个工具结果、文件工件、需求版本和恢复动作都进入同一可重放证据链。即使执行者发生变化，逻辑义务和完成标准仍然保持。

8.6 跨源研究案例分析

跨源研究任务共产生 7,168 次有效迁移，其中包括 3,551 次工具效果、3,562 次工件效果、12 次压缩恢复效果、7 次位置选择、7 次恢复、6 次路由和 8 次验证效果。任务仅访问冻结允许列表中的三个公开 HTTPS 来源，保存原始传输字节、请求身份、采集时间、内容摘要和最终引用。

网络与研究节点变化发生后，系统保留已验证来源及其摘要，拒绝陈旧或冲突事实进入策略，最终报告能够追溯到 RFC 9110、IANA HTTP 状态码登记和 MDN HTTP 状态资料。该案例验证了从采集、证据保存、记忆消费到最终报告的因果闭环。

8.7 指标与对照计划

表 5 论文评测指标及其约束

研究问题	主要指标	防止误判的配套指标
任务完成	可验证完成率、义务覆盖率	失败运行、验证器类型、样本量
拓扑与通信	通信字节、Token、边稀疏度	证据利用率、拓扑抖动、关键路径覆盖
记忆连续性	关键事实召回、未决义务保持	陈旧事实注入、来源完整性、重复工件
故障恢复	恢复率、恢复时延	迟到提交、重复副作用、非法位置迁移
资源效率	墙钟时间、Token、费用	任务成功、隐私和权限违规

下一轮正式对照实验将使用 Single Agent、Static Chain、Static Star、Broadcast 和 ZYRA Full 五种模式，并分别关闭环境修正、贡献剪枝、通信预算、连续性门和租约恢复机制。所有变体必须使用同一任务输入、模型族、预算、权限、故障计划和终止验证器。

只有完成新的端到端在线运行并达到足够样本量后，论文才会报告相对提升百分比、置信区间和统计显著性。本版结果不从冻结事件档案的机制重放推导开放环境中的总体成功率。

8.8 结果边界

当前结果支持以下结论：ZYRA 能够在两个指定、不同领域的长程任务中，在无人干预条件下处理需求变化和多类故障，维持规范状态、记忆和工件连续性，并产生通过独立验证的最终交付。当前结果尚不能证明某组拓扑参数在所有任务上最优，也不能证明系统对未见模型、硬件和行业场景具有统计意义上的普遍优势。

9 应用价值与可扩展性

9.1 面向复杂知识工作的数字劳动力

ZYRA 的直接应用价值在于把需要持续规划、工具操作、证据核验和异常处理的复杂知识工作转化为可观察、可恢复的执行过程。与一次性生成结果不同，系统保留任务如何推进、何处失败、如何恢复以及为何能够结束的完整证据，可用于软件交付、技术研究、运维处置、数据分析和合规材料生成等场景。

在软件工程中，系统可以协调代码、测试、浏览器和验证 Worker；在研究任务中，可以协调来源采集、事实核验、分析和写作角色；在终端生态中，可以根据隐私和时延选择本地、边缘或云端执行。能力注册与任务图分离使新增领域 Worker 不需要重写整个控制骨架。

9.2 用户体验与治理

面向普通用户，系统以自然语言目标为入口，隐藏大部分内部事件；面向评审和运营人员，Web 工作台提供拓扑、权限、工件、验证和恢复证据。运行中退出界面不会破坏后台任务，用户可以重新连接并恢复同一会话。权限请求、文件变化和最终交付采用分层展示，避免把内部诊断直接暴露为产品信息。

9.3 社会与经济价值

复杂任务自动化的价值不在于替代某一个操作步骤，而在于降低跨工具、跨角色和跨时间协调所需的智力成本。可验证、可恢复的运行时能够减少重复劳动、降低长任务中断后的重做成本，并使自动化结果更容易审计和复用。其商业化形态可以是面向行业的 Agent 运行平台，也可以作为终端、边缘与云端 AI 能力的统一调度底座。

10 局限与结论

10.1 当前局限

第一，当前拓扑策略采用确定性有界搜索和规则评分，便于复现和审计，但参数仍依赖任务与部署环境，不能视为跨领域最优策略。未来可使用历史运行轨迹校准软评分，同时继续保留硬约束投影。

第二，中心化状态托管减少了状态分叉，也可能形成提交与路由瓶颈。当前事件、检查点和幂等机制支持进程级恢复，但不等同于已经实现跨地域多副本共识。

第三，并非所有业务目标都能够完全形式化。代码测试、结构约束和摘要校验容易确定验证，审美判断和开放研究结论仍可能需要模型或人工评价。系统需要区分确定性证据、外部事实、模型评价和人工评价。

第四，事件日志和工件保留原始事实，但自然语言摘要仍可能产生语义损失。连续性门能够检查来源、版本和冲突，不能证明摘要保存了所有潜在相关细节。

第五，现有封存证据覆盖两个跨领域任务，仍不足以代表全部行业、模型和硬件。后续需要扩大任务样本，补充完整在线基线、消融、跨模型和真实跨设备实验。

10.2 结论

本文研究了超长程、动态和资源异构环境中的多智能体任务执行。核心困难不只是模型能否产生下一步，而是系统能否在需求变化、上下文压缩、节点故障和资源波动下持续维护同一个目标、任务状态与证据标准。

ZYRA 以“语义提议与状态裁决分离”为核心，形成中心化控制平面与去中心化异构执行相结合的体系。逻辑任务图维持稳定义务，动态协作图根据任务、环境和历史贡献在线调整；结构化消息和工件引用减少无关广播；事件日志、分层记忆、受保护压缩与连续性门维持长期状态；租约、代际、幂等和检查点使故障恢复不会接受旧结果或重复副作用。

两个封存跨领域任务表明，该系统能够在指定条件下跨越数千次规范状态迁移，接受需求变化和节点失效，并在无人干预下形成可验证交付。结果支持一种系统架构判断：长程群体智能的可靠性来自开放语义推理与确定性工程控制的协同，而不是单纯增加 Agent 数量或模型上下文长度。

参考文献

- [1] Yao S, Zhao J, Yu D, et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR, 2023.
- [2] Shinn N, Cassano F, Berman E, et al. Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS, 2023.
- [3] Wang G, Xie Y, Jiang Y, et al. Voyager: An Open-Ended Embodied Agent with Large Language Models. 2023.
- [4] Wu Q, Bansal G, Zhang J, et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations. COLM, 2024.
- [5] Zhuge M, Wang W, Kirsch L, et al. GPTSwarm: Language Agents as Optimizable Graphs. ICML, 2024.
- [6] Li S, Liu Y, Wen Q, Zhang C, Pan S. Assemble Your Crew: Automatic Multi-agent Communication Topology Design via Autoregressive Graph Generation. AAAI, 2026.
- [7] Wu T, Li Y, Tang Z, et al. CARD: Towards Conditional Design of Multi-Agent Topological Structures. ICLR, 2026.
- [8] Zhang G, Yue Y, Li Z, et al. Cut the Crap: An Economical Communication Pipeline for LLM-based Multi-Agent Systems. ICLR, 2025.
- [9] Lewis P, Perez E, Piktus A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS, 2020.
- [10] Park J S, O'Brien J, Cai C J, et al. Generative Agents: Interactive Simulacra of Human Behavior. UIST, 2023.
- [11] Packer C, Wooders S, Lin K, et al. MemGPT: Towards LLMs as Operating Systems. 2023.
- [12] Jiang H, Wu Q, Lin C Y, Yang Y, Qiu L. LLMingua: Compressing Prompts for Accelerated Inference of Large Language Models. EMNLP, 2023.
- [13] Liu B, Jiang Y, Zhang X, et al. LLM+P: Empowering Large Language Models with Optimal Planning Proficiency. 2023.
- [14] Olausson T, Gu A, Lipkin B, et al. LINC: A Neurosymbolic Approach for Logical Reasoning by Combining Language Models with First-Order Logic Provers. EMNLP, 2023.
- [15] Chandy K M, Lamport L. Distributed Snapshots: Determining Global States of Distributed Systems. ACM Transactions on Computer Systems, 3(1):63-75, 1985.
- [16] Kang Y, Hauswald J, Gao C, et al. Neurosurgeon: Collaborative Intelligence Between the Cloud and Mobile Edge. ASPLOS, 2017.
- [17] Li E, Zhou Z, Chen X. Edge Intelligence: On-Demand Deep Learning Model Co-Inference with Device-Edge Synergy. 2018.
- [18] 荣耀终端股份有限公司. 面向超长程复杂任务的动态异构群体智能架构与深度协同推理技术比赛方案. 2026.

附录 A 核心算法与复杂度补充

A.1 任务完成门

任务成功不依赖某个 Agent 的自然语言声明。当前需求版本下所有必要义务必须通过完成谓词，必需工件存在且摘要一致，关键记忆通过连续性门，同时不存在仍开放的关键租约和未确认副作用。

$$\text{Success}(X(T)) \Leftrightarrow (\forall o(i) \in \text{Orequired}(T), p(i, A(T)) = 1) \wedge \text{RequiredArtifacts} \wedge \text{ContinuityGate} \wedge \text{NoOpenCriticalLease}$$

(12)

预算耗尽、用户取消、硬约束不可满足或故障不可恢复时，系统进入阻塞、取消或失败终态，并保留最后可信状态与恢复条件。

A.2 通信指标

通信机制同时报告路由字节相对于广播基线的比例、证据利用率、单位验证义务成本和义务覆盖率。通信成本下降不能以验证完成率下降为代价。若只发送很少消息却使关键义务无法闭合，应被视为失败而非优化。

A.3 记忆检索

记忆检索以规范记录为水化来源。词法、向量、重要度和新鲜度可以提供候选排序信号，多样性重排减少近重复结果。向量服务不可用时允许退化到词法检索，但必须记录退化原因；索引中缺失或摘要不一致的记录不得进入关键上下文。

A.4 恢复不变量

任一逻辑义务在同一时刻至多存在一个拥有有效提交权的物理尝试。新尝试不得削弱原能力、位置、隐私和资源要求，除非显式需求修订改变了约束。旧执行者的迟到回执、输入摘要不一致的幂等请求和无法确认的副作用均不能直接提交。

附录 B 工程实现与部署边界

B.1 产品入口

CLI 是默认执行入口，Web 是观察与复杂治理入口，API 服务托管任务、会话、事件、权限、工件和运行时接口。三者共享同一规范状态，不存在由 Web 或 CLI 私自维护的第二份任务事实。

B.2 运行时职责

Python 侧负责核心模型、任务图、图状态托管、拓扑策略、记忆、Worker 注册、调度、恢复和 API；TypeScript 侧负责产品 CLI、Web、Provider 控制以及代码 Worker 的模型工具循环。运行时之间通过类型化消息与事件连接。

B.3 部署要求

系统要求 Python 3.12 或更高版本、Node.js 22 和 Bun 1.2.15。正式提交包应包含依赖锁定文件、示例环境配置、一键安装和启动脚本、健康检查及最小演示任务。真实模型凭据由部署者通过环境变量提供，不进入源代码、论文或运行证据。

B.4 技术细节披露边界

正文公开核心机制、输入输出、不变量、伪代码和复杂度；附录只保留理解复现所需的运行时职责和部署边界。精确评分权重、阈值、内部 API、数据库结构、安全 Token、物理端点和本机路径不在论文中披露，由受控配置和部署材料单独管理。

附录 C 证据与复现说明

C.1 权威封存证据

本文第 8 章的两个场景结果来自 P2-S06-02 attempt 08。软件工程任务标识为 run_bc378ef6b364，跨源研究任务标识为 run_897c2d9ea4a9。独立验证报告摘要为 051b3e6de622a17c15e3d08573363f13b4f4c79ffc88d447a17b5c55c3cce084。

该证据绑定候选代码版本、任务输入、配置、故障计划、原始规范事件、迁移索引、最终工件和验证器。独立验证器从原始事件重新计算状态迁移和摘要链，不采信运行器自报计数。

C.2 实验复现原则

每组实验应冻结代码版本、任务版本、模型及参数、上下文预算、拓扑配置、权限策略、硬件与网络配置、故障注入时刻和独立验证器。每次运行至少保存规范事件日志、任务图与协作图版本、Worker 派发和租约回执、资源统计、最终工件、验证报告以及失败与排除说明。

C.3 后续对照实验

后续实验将对每个基线和消融执行新的端到端在线运行，而不是只重放同一因果档案。成功率报告样本量和区间，Token、时延、字节和成本报告均值、中位数、离散度和分位数。失败运行是实验结果的一部分，只有预先定义的基础设施故障或证据损坏才能排除并需要公开说明。